p1066R0
How to catch an exception_ptr without even try-ing

Published Proposal, 7 May 2018

This version:
wg21.link/P1066

Author:
Mathias Stearn (MongoDB)

Audience:
EWG, LEWG

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Latest Version:
Click here

Source:
No real reason to click here

Abstract

Adding facilities to inspect and handle std::exception_ptr without throwing and catching. This mechanism should work even in environments that use -fno-exceptions.

Table of Contents

§ 1. Introduction

`std::exception_ptr` is a weird beast. Unlike the other `_ptr` types it is completely type erased, similar to a hypothetical `std::any_ptr`. Presumably because of this, it is the only `_ptr` type to offer no `get()` method, since it wouldn't know what type to return. You might even say it should take [N4282]'s tile as *"The World's Dumbest Smart Pointer"* since the only way to dereference it is by throwing!

This proposal suggests adding methods to `std::exception_ptr` to directly access the pointed-to exception in a type-safe manner. On standard libraries that implement `std::make_exception_ptr` by direct construction rather than using `try`/`throw`/`catch`/`current_exeption()` (currently MS-STL and libstdc++, but not libc++), it should now be possible to both create and consume a `std::exception_ptr` with full fidelity even with exceptions disabled. This should ease interoperability between codebases and libraries that choose to use exceptions and those that do not.

I have a proof of implementability without ABI breakage here. It is an implementation of the proposed methods for both MSVC and libstdc++. On Linux, it is **385** times faster than using `std::rethrow_exception` and `catch` as is needed today. On Windows, it is *only* 10 times faster due to needing to trap into the kernel to call ***DecodePointer()***. I haven't tested on libc++ or with the special ARM EH ABI, but based on my reading of those implementations, the same strategy should work fine. Pull requests welcome!

§ 2. Proposal

This is an informal descriptions of the methods I propose adding to `exception_ptr`. I don't have fully fleshed out proposed wording yet.

All pointers returned by this API are valid until this `exception_ptr` is destroyed or assigned over. Copying or moving the `exception_ptr` will not extend the validity of the pointers.

Calling any of these methods on a null `exception_ptr` is UB.

§ 2.1. High-level API

§ 2.1.1. *handle()*

```
template <typename... Handlers>
/*see below*/ handle(Handlers&&... handlers) const;
bool handle() const { return false; }
```

Handles the contained exception as if there were a sequence of `catch` blocks that catch the argument type of each handler. The argument type is determined in a similar way to the `function(Handler)::argument` template deduction guide to

detect the first and only argument type of a Callable object. However, it must also detect an `R(...)` callable as the natural analog of the `catch(...)` catch-all. If present, the catch-all must be the last handler.

The return type of `handle()` is the natural meaning of combining the return types from all handers and making it optional to express nothing being caught. More formally:

```
using CommonReturnType = common_type_t<result_of_t<Handlers>...>;
using ReturnType = conditional_t<is_void_v<CommonReturnType>,
                                 bool,
                                 optional<CommonReturnType>>;
```

If none of the handlers match the contained exception type, the call to `handle()` returns either `false` or an empty `optional`. If any handler matches, it returns either `true` or an `optional` constructed from the handler's return value.

This API is inspired by folly::exception_wrapper. It can be implemented in the standard today, albeit without the performance benefits. Additionally, I think the valid lifetimes of the references would be shorter than is proposed here.

§ 2.1.2. *handle_or_terminate()*

```
template <typename... Handlers>
common_type_t<result_of_t<Handlers>...>
handle_or_terminate(Handlers&&... handlers) const;
```

Similar to handle(), but calls terminate_with_active() if no handler matches the current exception. Unwraps the return type since if it returns, a handler must have matched.

§ 2.1.3. *try_catch()*

```
template <typename T> requires is_reference_v<T>
add_pointer_t<T> try_catch() const;

template <typename T> requires is_pointer_v<T>
optional<T> try_catch() const;
```

If the contained exception is catchable by a `catch(T)` block, returns either a pointer to the exception if `T` is a reference or an `optional` containing the caught pointer if `T` is a pointer. If the `catch(T)` block would not catch the exception, returns `nullopt`/`nullptr`.

The pointer case is a bit odd and throwing/catching pointer is fairly rare so it could use some explanation for why it is both different and the same as the reference case. They have different return types because returning `T*` is impossible since the exception holds a `U*` and `T` may be `X*`, and there is no `X*` object to return a pointer to. Returning `T`/`X*` in this case would also be incorrect because there would be no way to distinguish a thrown null pointer from a type mismatch. Luckily, `optional<T>` has the same access API as `T*` so even though the return types of these functions are different, consumers can treat them the same:

```
if (auto ex = ex_ptr.try_catch<CatchT>()) {
    use(*ex);
}
```

2.1.4. *terminate_with_active()*

```
[[noreturn]] void terminate_with_active() const noexcept;
```

Equivalent to:

```
try {
    rethrow_exception(*this);
} catch (...) {
    terminate();
}
```

Invokes the terminate handler with the contained exception active to allow it to provide useful information for debugging.

> Note: I believe this definition is exactly equivalent to just calling `rethrow_exception` inside a `noexcept` context, but I wrote it that way to avoid bugs we've seen with `noexcept`, particularly when used for this purpose.

2.2. Low-level API

This is the low-level API that is intended for library authors building their own high-level API, rather than direct use by end users.

2.2.1. *type()*

```
type_info* type() const;
```

Returns the `type_info` corresponding to the exception held by this `exception_ptr`.

2.2.2. *get_raw_ptr()*

```
void* get_raw_ptr() const;
```

Returns the address of the exception held by this `exception_ptr`. It is a pointer to the type described by type(), so the caller will need to cast it to something compatible in order to use this.

3. Use Cases

3.1. Lippincott Functions

Here is a lightly-modified example from our code base that shows a **100x** speedup on Linux:

Now	With this proposal

`Status exceptionToStatus() noexcept {`
`  try {`
`  throw;`
`  } catch (const DBException& ex) {`
`  return ex.toStatus();`
`  } catch (const std::exception& ex) {`
`  return Status(ErrorCodes::UnknownError,`
`  ex.what());`
`  } catch (...) {`
`  std::terminate();`
`  }`
`}`
	`Status exceptionToStatus() noexcept {`
	`  return std::current_exeption().handle_or_termina`
	`  [] (const DBException& ex) {`
	`  return ex.toStatus();`
	`  },`
	`  [] (const std::exception& ex) {`
	`  return Status(ErrorCodes::UnknownError,`
	`  ex.what());`
	`  });`
	`}`

1892ns	18ns

Windows shows *only* a 2x speedup (1488ns -> 1744ns) due to the DecodePointer() issue mentioned above (160ns), as well as `std::current_exeption()` being even slower (575ns). Essentially all of the runtime is in those two functions. (Windows tests were conducted on different hardware, so the results cannot be directly compared to Linux.)

§ 3.2. Terminate Handlers

It is common practice to use terminate handlers to provide useful debugging information about the failure. libstdc++ has a default handler that prints the type of the thrown exception using it's privileged access to the EH internals. Unfortunately, there is no way to do that in the general case in a user-defined terminate handler. type() makes that information available from `current_exception` in a portable way.

As a historical sidenote, that message from the terminate handler is what initially got me looking into the ABI for exception handling to figure out how that was done. If whoever wrote that code is reading this, thanks!

§ 3.3. `std::expected` and Similar Types

These types become more useful with the ability to interact with the exception directly without rethrowing.

§ 3.4. Error Handling in Futures

Error handling in Future chains naturally involves passing error objects into callbacks as arguments. There has been an active discussion around avoiding `exception_ptr` due to these issues. In addition to the direct performance benefits of avoiding the unwinder, this also makes it easier to provide nicer APIs like `future.catch_error([](SomeExceptionType&) {})` that only invokes the user's callback when the types match. This has a secondary benefit of avoiding a trip through the executor's scheduler if the callback isn't to be called. It also has the (unconfirmed) potential of being implementable on GPUs which don't currently support exceptions.

§ 4. It sounds like you just want faster exception handling. Why isn't this just a QoI issue?

It has been 30 years since C++98 was finalized. Compilers seem to actively avoid optimizing for speed in codepaths that involve throwing, which is usually a good choice. But this means that even in trivial cases, they aren't able to work their usual magic. Here is an example function that should be reduced to a constant value of `0`, but instead goes through the full `throw`/`catch` process on all 3 major compilers. On my Linux desktop that means it takes 5600 cycles, when it should take none.

```
int shouldBeTrivial() {
    try {
        throw 0;
    } catch (int ex) {
        return ex;
    }
    return 1;
};
```

Given how universal the poor handling of exceptions is, I don't see much hope for improvement to the extent proposed here in the realistic, non-trivial cases. Additionally, I think the ergonomics are better if the exception object is made directly available than requiring the `try`/`catch` blocks.

§ 5. Related Future Work

These are related ideas that are **not** part of this proposal. If this proposal is well received, I'd like a straw poll of these to gauge the interest for future proposals.

§ 5.1. Support dynamic `dynamic_cast` using `type_info`

My initial implementation plan called for adding casting facilities to `type_info` and building the try_catch() logic on top of that. Since it ended up being the wrong route for the MSVC ABI, I abandoned that plan, but it still provides useful independent functionality. Something like adding the following methods on `type_info`:

```
template <typename T>
bool convertable_to()

template <typename T>
T* dynamic_dynamic_cast(void* ptr);
```

§ 5.2. `dynamic_any_cast`

Currently `any` only supports exact-match casting. Using a similarly enhanced `type_info` it should be able to support more flexible extractions.

§ 5.3. Less copies in `std::make_exception_ptr(E e)`

I noticed that the current definition takes the exception by value and is defined as copying it. Should it take the exception object by forwarding reference and forward it? If consensus is yes, I'd be happy to either submit a new paper or just add that to the proposed wording here.

§ Index

§ Terms defined by this specification

type(), in §2.2.1

§ References

§ Informative References

[N4282]
Walter E. Brown. A Proposal for the World's Dumbest Smart Pointer, v4. 7 November 2014. URL: https://wg21.link/n4282